Processing and Analysis of Biological Data

Chapter 5. Generalized Linear Models (GLM)

Øystein H. Opedal, Department of Biology, Lund University

2025-12-14

Introduction

Generalized linear models (GLMs) add flexibility to the linear model by allowing deviations from the usual assumption of normally distributed residuals. Briefly, a GLM consists of the familiar linear predictor of a linear model (often denoted as η),

$\eta = \beta_0 + \Sigma_j x_{ij}\beta_j + \epsilon_i$

and a link function, g, that places the predictor on a Gaussian (normally-distributed) scale.

$$y = g^{-1}(\eta)$$

Before going into details about GLMs, we need to recall some basics about the most common error distributions used in biological data analyses.

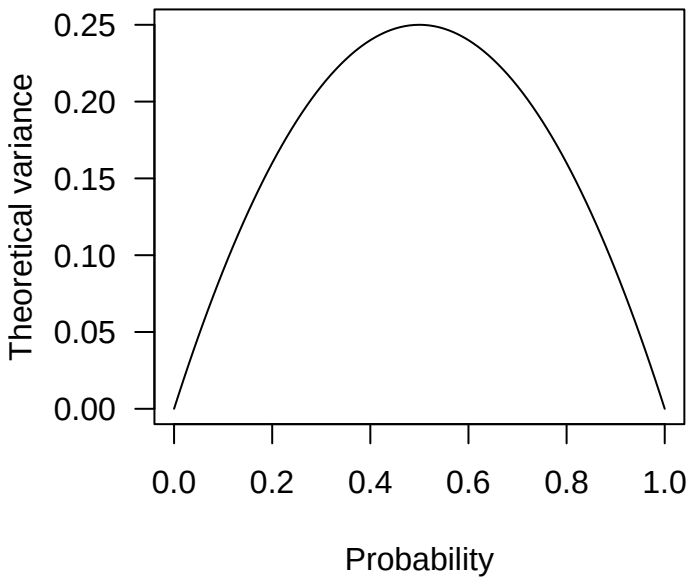
The Binomial distribution

The binomial distribution has two parameters, n and p, and summarizes a set of so-called Bernoulli trials with two possible outcomes, yes (1) or no (0). When we have performed more than one trial, we can compute the proportion p of the n trials with a positive outcome.

The theoretical variance of the binomial distribution is given by

$\sigma^2 = np(1-p)$

```
x = seq(from=0, to=1, by=0.01)
v_b = x*(1-x) #Binomial variance
plot(x, v_b, type="l", xlab="Probability", ylab="Theoretical variance", las=1)
```



This is important to keep in mind, because it affects how we can compare the (proportional) variation of variables measures as proportions. If e.g. one population has a mean of 0.5, if will be expected to be much more variable than a second population with a mean of 0.1, just because there is less opportunity to vary. This is a now well-recognized problem e.g. in demography research, where the interest is often in comparing the extent of variation in life-history traits measured as proportions, such as germination or survival. One proposed solution is to scale the observed CV by the maximum based on the theoretical variance, but a perhaps simpler approach is to transform the proportional data in a way that makes them approach a normal distribution.

One previously popular but now not recommended (see Warton & Hui 2011 Ecology) transformation is the so-called arcsine square-root transformation,

$x' = \arcsin(\sqrt{x})$

A more meaningful transformation is the logit or log odds transformation

$logit(x) = log(\frac{x}{1-x})$

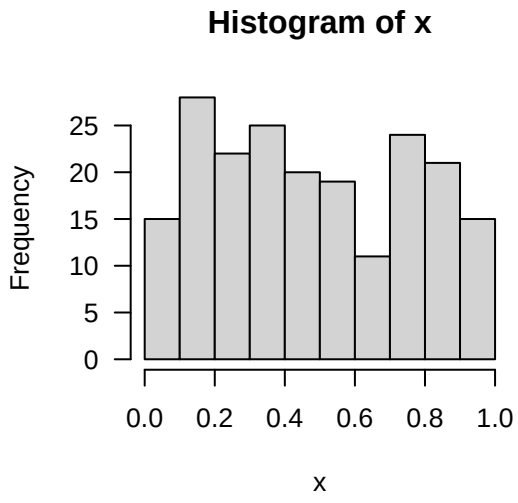
In the following example we are using some functions. This may be a good time to visit the Appendix introducing some basics on writing functions. Note that when functions are given on a single line, we can skip the special curvy brackets (`{}`).

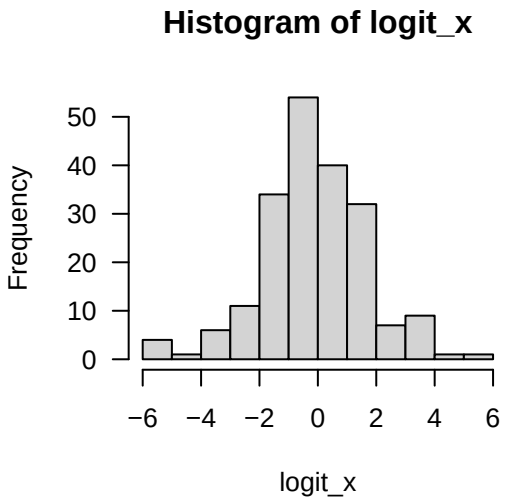
```
logit = function(x) log(x/(1-x))
invlogit = function(x) 1/(1+exp(-x))

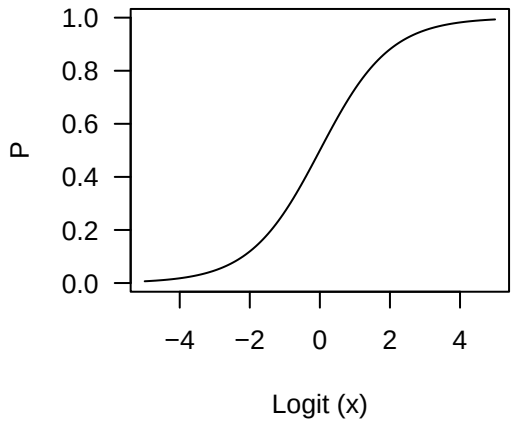
x = runif(200)
logit_x = logit(x)

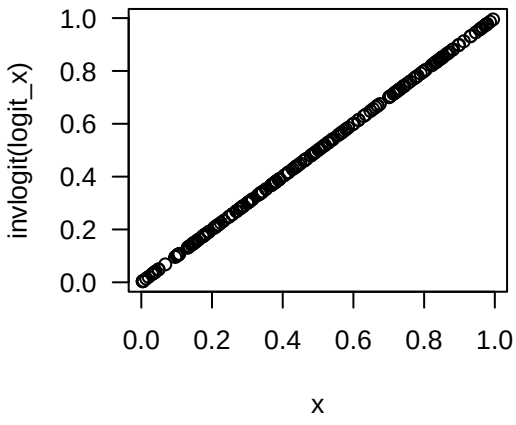
par(mfrow=c(2,2))
hist(x, las=1)
hist(logit_x, las=1)
```

```
xx = seq(-5, 5, 0.01)
plot(xx, invlogit(xx), type="l", las=1,
     xlab="Logit (x)",
     ylab="P")
plot(x, invlogit(logit_x), las=1)
```



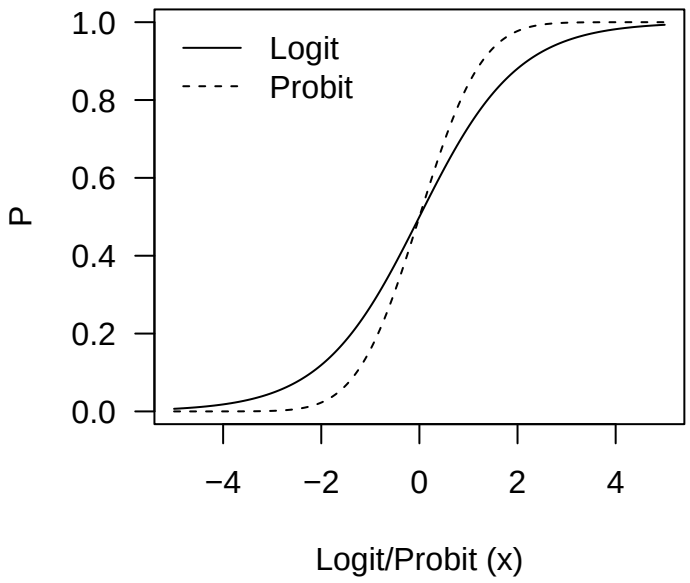






The logit transformation is the most common *link function* in a Generalized Linear Model with binomial errors. A similar alternative is the so-called probit link, which corresponds to the quantile distribution of the standard normal distribution (which is why we can use the `pnorm` function to compute the inverse).

```
plot(xx, invlogit(xx), type="l", las=1,
     xlab="Logit/Probit (x)",
     ylab="P")
lines(xx, pnorm(xx), lty=2)
legend("topleft", legend=c("Logit", "Probit"),
       lty=c(1,2), bty="n")
```



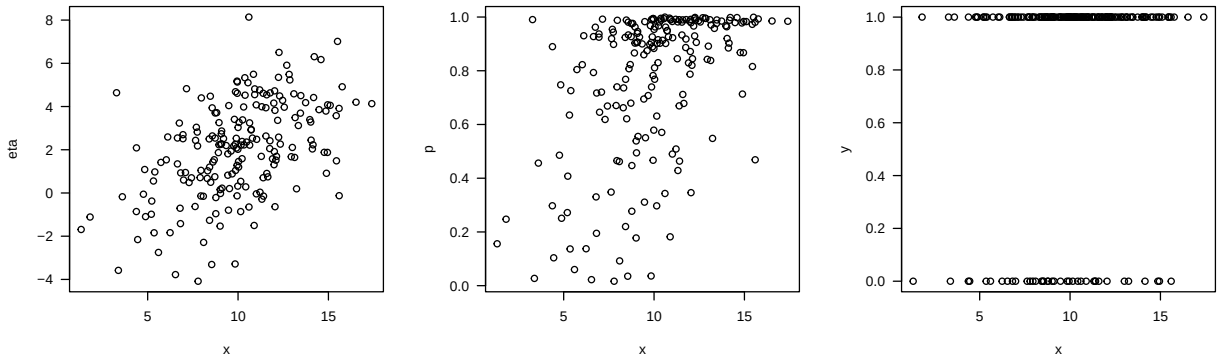
Logistic regression

Given the properties of the logit and similar link functions outlined above, it follows that logistic regression (GLM with binomial errors) is well suited for analysing binary data (or proportions).

Below, we simulate data by first formulating a linear predictor η, then transforming the predicted values into probabilities (through the inverse-logit transformation), and finally binarizing the data by sampling from the binomial distribution. The last step adds additional realistic uncertainty by accounting for the stochasticity of the observation process (alhough the underlying probabilities range continuously from 0 to 1, we only "see" ones and zeros.

```
x = rnorm(200, 10, 3)
eta = -2 + 0.4*x + rnorm(200, 0, 2)
p = invlogit(eta)
y = rbinom(200, 1, p)

par(mfrow=c(1,3))
plot(x, eta, las=1)
plot(x, p, las=1)
plot(x, y, las=1)
```



To fit the generalized linear model, we use the `glm` function and specify the error family (binomial) and link function (here *logit*).

```
m = glm(y~x, family=binomial(link="logit"))
summary(m)
```

```
##
## Call:
## glm(formula = y ~ x, family = binomial(link = "logit"))
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.23229    0.60787   0.382   0.7024
## x            0.12195    0.06163   1.979   0.0478 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 197.36  on 199  degrees of freedom
## Residual deviance: 193.36  on 198  degrees of freedom
## AIC: 197.36
##
## Number of Fisher Scoring iterations: 4
```

The summary table includes, as always, a lot of information. The first thing to be aware of is that when we are fitting a GLM, we obtain the parameter estimates on the link scale (here logit). Note that the parameter estimates are not too far from those we used to define the linear predictor η when we simulated the data. These values are meaningful as such, and if the predictor variable has units of *mm*, the slopes have units of *log odds* mm^{-1}.

To interpret the results biologically and to represent them in graphs, it can be useful to back-transform the predicted values to the probability scale. For example, we can ask how much the probability changes for a standard deviation increase in the predictor variable (although note that this is no longer a linear transform, so the consequences of increasing and decreasing the predictor by one standard deviation may be different).

Recall from the previous section that a log odds of 0 corresponds to a probability of 0.5 ($log(\frac{0.5}{1-0.5}) = log(1) = 0$). If we solve the model equation (the linear predictor) for 0, we can thus obtain the predictor value corresponding to a probability of 0.5, which is often a relevant benchmark. A classic example comes

from toxicology, where it is often of interest to compute a "lethal dose 50 (LD_{50}), the dose of a substance associated with a 50% probability of death.

$0 = \beta_0 + \beta_x x$

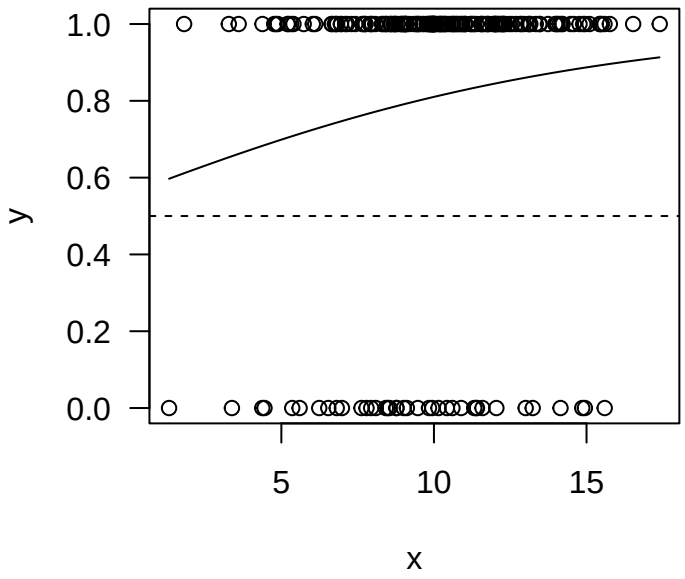
$\frac{-\beta_0}{\beta_x} = x$

EXERCISE: Replicate the plot below. To produce a regression line, we define some new x-values that spans the data range along the x-axis, then obtain predicted values $\hat{y}$ (using the model coefficients), and finally transform these values to the probability scale to obtain the predicted probabilities $\hat{p}$.

```
coefs = summary(m)$coef

x_pred = seq(from=min(x), to=max(x), by=0.01)
y_hat = coefs[1,1] + coefs[2,1]*x_pred
p_hat = invlogit(y_hat)
```

Compute the value of x corresponding to a probability of 0.5, and add lines to the plot to illustrate the results.



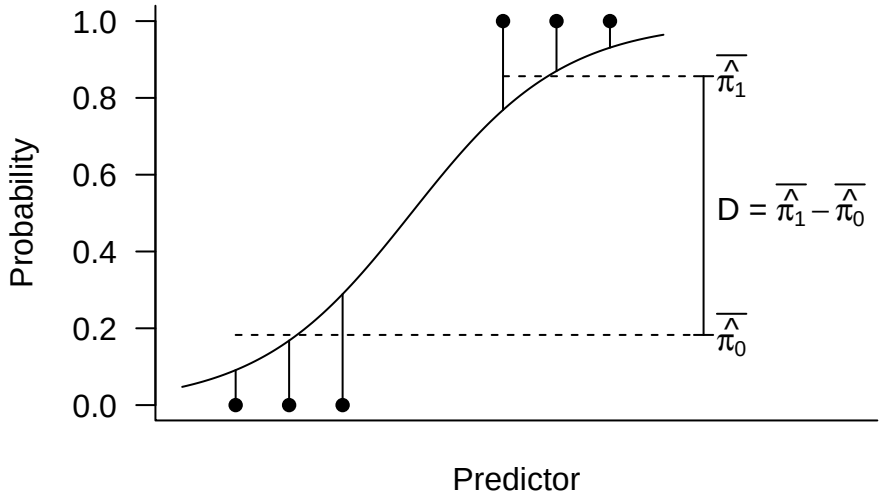
The GLM summary table does not provide an r^2 value, because the normal r^2 does not work for logistic regression. There are however several 'Pseudo-r^2' available, typically based on comparing the likelihood of the model to that of a null model (a similar model but with only an intercept). The `MuMIn` package provides some such measures. The package returns both a marginal (r^2_M) and a conditional r^2 (r^2_C). For now, we will focus only on the former, which is akin to the usual r^2 from linear models. We will return to the (r^2_C) when discussing mixed models. Because it is not entirely obvious how to compute the residual variance in these models (and hence the r^2), the package provides several options. Perhaps the most flexible of these is estimating the variance through the delta method.

```
library(MuMIn)
r.squaredGLMM(m)
```

```
##                    R2m        R2c
## theoretical 0.03766394 0.03766394
## delta       0.01981151 0.01981151
```

Another possibility for logistic regression is to compute the coefficient of discrimination, or Tjur's D. Indeed, there are several ways to evaluate the performance of a statistical model, and in a logistic regression it makes sense to ask how well the model discriminates between true positives and negatives in the data.

The coefficient of discrimination is defined as $D = \bar{\hat{\pi}}_1 - \bar{\hat{\pi}}_0$, and is computed by comparing the predicted probability for those data points that are successes or positives (1s; $\bar{\hat{\pi}}_1$) to the predicted probability for those data points that are failures or negatives (0s; $\bar{\hat{\pi}}_1$).




```
y_hat = coefs[1,1] + coefs[2,1]*x
p_hat = invlogit(y_hat)

mean(p_hat[which(y==1)]) - mean(p_hat[which(y==0)])
```

```
## [1] 0.02169169
```

Some final notes on fitting binomial GLMs. There are three ways to formulate these models in R. In the example above, the data were 0's and 1's, and we could specify the model simply as

```
glm(y ~ x, family=binomial(link="logit"))
```

When each observation is based on more than one trial, we can formulate the model in two ways. The first is

```
glm(y ~ x, family=binomial(link="logit"), weights=n)
```

where `y` is the proportion of successes, and `n` is the number of trials. The second method is to fit a two-column matrix as response variable, where the first colomn is the number of successes, and the second column is the number of failures, i.e. `y = cbind(successes, failures)`. The model formula is then

```
glm(cbind(successes, failures) ~ x, family=binomial(link="logit"))
```

Data exercise: seed germination

The following data are from a study investigating patterns of seed dormancy in a plant. In this plant species, seeds needs to stay in dry conditions for a specific amount of time before they are ready to germinate, a process known as 'after-ripening'. Once the seeds are ripe they will normally germinate when exposed to favourable (moist) conditions.

After different duration of after-ripening, the seeds were sown on moist soil and their germination success (proportion of seeds germinated) recorded. The seeds came from four different populations, and were weighed (in *mg*) prior to sowing.

The variables in the data are as follows:

- `pop` = Population ID
- `mother` = Maternal plant ID
- `crossID` = Unique cross identifier
- `blocktray` = Sowing tray (experimental block)
- `timetosowing` = Time in days from seed dispersal to watering
- `MCseed` = Population-mean-centered seed mass in mg
- `nseed` = Number of seeds sown
- `germ2` = Proportion of seeds germinated

Analyse the data to estimate the pattern of germination success in response to variation in the duration of after-ripening. Are the patterns similar in different populations? Are there other factors affecting germination success? Produce relevant summary statistics, parameter estimates, and graphs.

```
dat = read.csv("datasets/dormancy/dormancy.csv")
names(dat)
```

```
## [1] "pop"          "mother"       "crossID"      "blocktray"    "timetosowing"
## [6] "MCseed"       "nseed"        "germ2"
```

As a suggested start, the following lines fit a simple model to data from one population using two different methods. Note that the model fit is the same (the log Likelihood of the two models is identical).

```
subdat = dat[dat$pop=="CC",]

germ = subdat$germ2 * subdat$nseed #Successes
notgerm = subdat$nseed - germ #Failures

mod1 = glm(cbind(germ, notgerm) ~ timetosowing, "binomial", data=subdat)
mod2 = glm(germ2 ~ timetosowing, "binomial", weights=nseed, data=subdat)
logLik(mod1) == logLik(mod2)
```

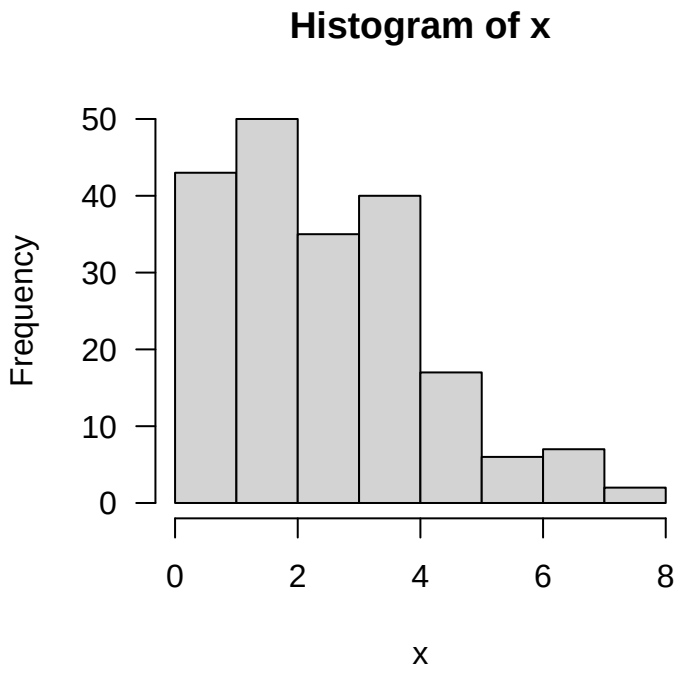
```
## [1] TRUE
```

Can you use the fitted models to estimate the duration of after-ripening required for the expected germination rate to be 0.5?

Poisson and negative-binomial regression

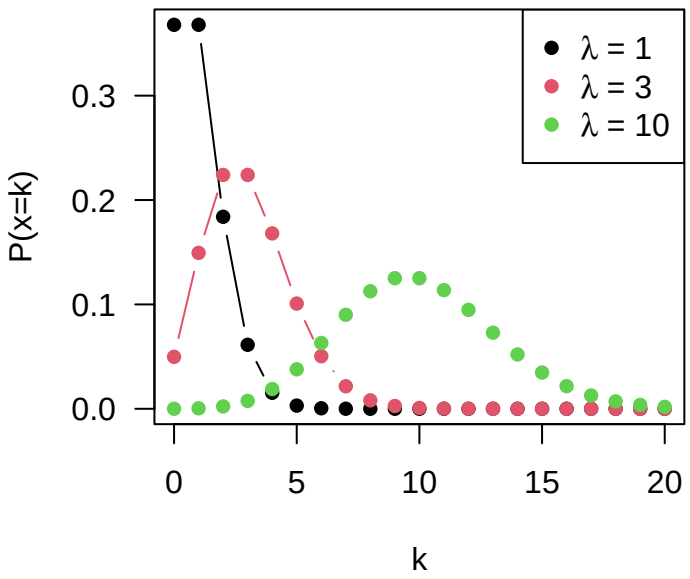
A second very common data type in biology is count data, which occurs when we have counted something. In ecological studies we often count individuals or species, and in evolutionary biology we often count e.g. the number of offspring. For such data, the data distribution is often skewed, and the variance tends to increase with the mean. The Poisson distribution is tailored for such data.

```
x = rpois(200, 3)
hist(x, las=1)
```



The Poisson distribution has a single parameter λ that determines both the mean and the variance, so that $E(X) = Var(X) = \lambda$. Thus, the variance increases linearly with the mean. The probability mass function changes quite dramatically for different values of λ. For low values the distribution is highly skewed, for high values the distribution approaches a Gaussian (normal) distribution. However, the variance is still constrained to be the same as the mean, which is not the case for the normal distribution.

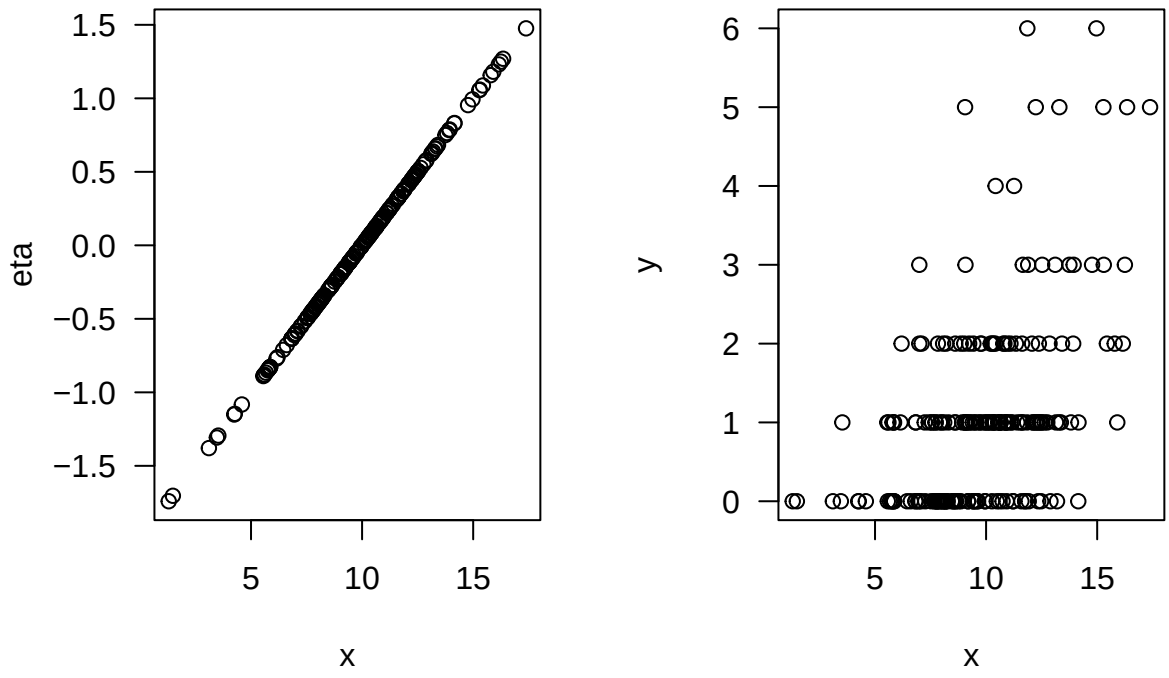
```
x = seq(0, 20, 1)
y = dpois(x, lambda=1)
plot(x,y, type="b", las=1, xlab="k", ylab="P(x=k)", pch=16, col=1)
points(x, dpois(x, lambda=3), type="b", pch=16, col=2)
points(x, dpois(x, lambda=10), type="b", pch=16, col=3)
legend("topright", col=1:3, pch=16,
       legend=c(expression(paste(lambda, " = 1")),
                expression(paste(lambda, " = 3")),
                expression(paste(lambda, " = 10"))))
```



The distribution of count data can sometimes by normalized through a log-transformation, and the log is indeed the link function of a Poisson regression model. The alternative method of log-transforming the data and then fitting a Gaussian model is problematic when there are zeros in the data. Adding a constant (e.g. 0.5 or 1) is sometimes an option, but is generally not recommended. A better option is to analyze the data in a GLM framework with Poisson-distributed errors and a log link function.

```
x = rnorm(200, 10, 3)
eta = -2 + 0.2*x
y = rpois(200, exp(eta))

par(mfrow=c(1,2))
plot(x, eta, las=1)
plot(x, y, las=1)
```




```
m = glm(y~x, family="poisson")
summary(m)
```

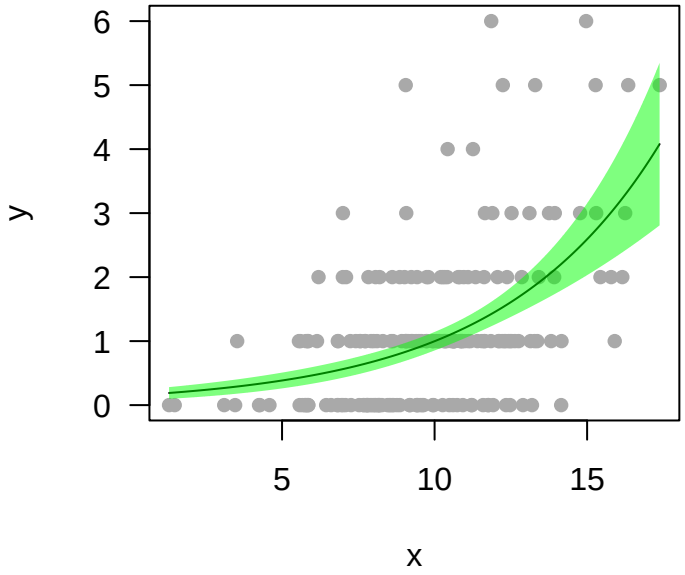
```
##
## Call:
## glm(formula = y ~ x, family = "poisson")
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.90913    0.27600  -6.917 4.61e-12 ***
## x            0.19077    0.02369   8.054 8.04e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 276.08  on 199  degrees of freedom
## Residual deviance: 211.19  on 198  degrees of freedom
## AIC: 515.85
##
## Number of Fisher Scoring iterations: 5
```

When interpreting Poisson regressions, there are several things to keep in mind. First, as in all GLMs, the parameters are reported on the link scale, here log. Recall that the link scale has useful proportional properties, so that the slope can be interpreted roughly as the proportional change in y per unit change in

x. To plot the fitted regression line, we have to back-transform the predicted values. This time we use the generic `predict` function to obtain the predicted values on the data scale, and to construct a 95% confidence polygon.

```
plot(x, y, las=1, col="darkgrey", pch=16)
xx = seq(min(x), max(x), 0.01)
y_hat = predict(m, newdata=list(x=xx), type="response", se.fit=T)
lines(xx, y_hat$fit)
#lines(xx, y_hat$fit+1.96*y_hat$se.fit, lty=2)
#lines(xx, y_hat$fit-1.96*y_hat$se.fit, lty=2)

polygon(c(xx, rev(xx)),
        c(y_hat$fit+1.96*y_hat$se.fit,
          rev(y_hat$fit-1.96*y_hat$se.fit)),
        col = rgb(0,1,0,.5), border = FALSE)
```



As in logistic regression the normal r^2 is not valid, and we can use e.g. the `r.squaredGLMM` function to obtain a Pseudo r^2. For a simple GLM an older Pseudo r^2 is

$$1 - \frac{Residual\ deviance}{Null\ deviance}$$

.

```
r.squaredGLMM(m)
```

```
## Warning: the null model is only correct if all the variables it uses are identical
## to those used in fitting the original model.
```

```
##                 R2m       R2c
## delta     0.2459542 0.2459542
## lognormal 0.3141419 0.3141419
## trigamma  0.1716958 0.1716958
```

```
1-(m$deviance/m$null.deviance)
```

```
## [1] 0.2350407
```

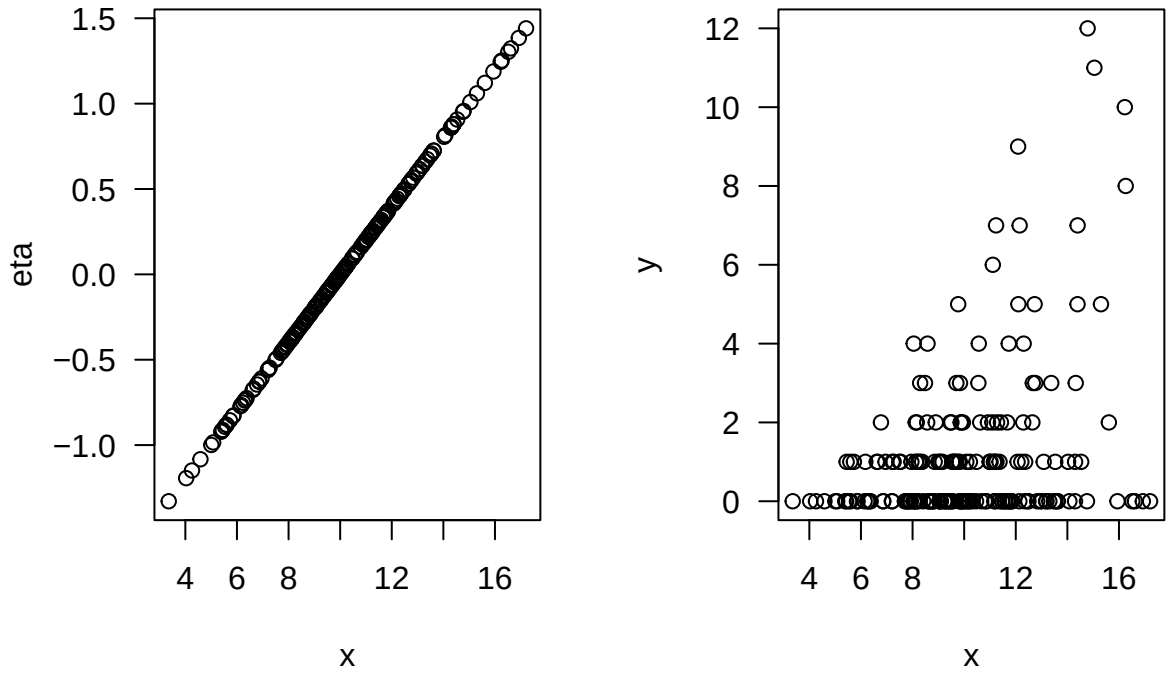
The *deviance* is a measure of how far our model is from a "perfect" or saturated model, i.e. one that perfectly explains the data. Fitting a GLM is done by maximizing the likelihood of a model, which is the same as minimizing the deviance. The deviance is defined mathematically as 2 times the difference in the log likelihood of the model and a saturated model, and can be used to compare alternative models. For example, the Pseudo r^2 above is based on comparing the (mis)fit of the focal model to a null model with only an intercept. If a model is as bad as a null model, the Pseudo r^2 will be 0. We can also use the deviance to compare other models, e.g. through likelihood ratio tests (more on that later).

Second, recall that the Poisson distribution has a single parameter λ determining both the mean and the variance. In real count data the variance often increase disproportionally compared to the mean, a phenomenon called *overdispersion*. Biologically, this occurs because we tend to observe multiple entities together. For example, in a survey of common eiders, we will often see either no eiders, or a lot of eiders.

We can quantify overdispersion in the data based on the fitted model by calculating the ratio of the residual deviance to the residual degrees of freedom. In the model above there is no serious overdispersion, because the residual deviance is only a little larger than the residual degrees of freedom. Let us construct an example with more serious overdispersion.

```
set.seed(1)
x = rnorm(200, 10, 3)
eta = -2 + 0.2*x
y = rnbinom(200, 1, mu=exp(eta))

par(mfrow=c(1,2))
plot(x, eta, las=1)
plot(x, y, las=1)
```




```
m = glm(y~x, family="poisson")
summary(m)
```

```
##
## Call:
## glm(formula = y ~ x, family = "poisson")
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -2.03980    0.27569  -7.399 1.37e-13 ***
## x            0.20177    0.02284   8.833  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 506.63  on 199  degrees of freedom
## Residual deviance: 429.12  on 198  degrees of freedom
## AIC: 675.26
##
## Number of Fisher Scoring iterations: 6
```

Here the overdispersion is serious, and we can not trust the model estimates. In this case, we need an alternative link function that allows the variance to increase more than the mean. The negative binomial distribution is a good option. The negative binomial is similar to the Poisson distribution, but includes an additional parameter modelling the disproportionate increase in variance with increasing mean.

```
library(MASS)
m = glm.nb(y~x)
```

```
summary(m)
```

```
##
## Call:
## glm.nb(formula = y ~ x, init.theta = 0.7408479419, link = log)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -2.19800    0.44356  -4.955 7.22e-07 ***
## x            0.21624    0.03989   5.420 5.95e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Negative Binomial(0.7408) family taken to be 1)
##
##     Null deviance: 214.23  on 199  degrees of freedom
## Residual deviance: 183.51  on 198  degrees of freedom
## AIC: 568.01
##
## Number of Fisher Scoring iterations: 1
##
##
##               Theta:  0.741
##           Std. Err.:  0.154
##
##  2 x log-likelihood:  -562.012
```

Data exercise: bee distribution

The following dataset includes the abundance of the bee species *Eulaema nigrita* in the Brazilian Atlantic forest, and a number of potential predictor variables including climate (mean annual temperature and precipitation, temperature and precipitation seasonality) and land use (proportion forest cover and land use heterogeneity defined as the Shannon diversity ($-\Sigma_i p_i ln p_i$) of local land-use classes).

The climate data from the *Worldclim* dataset are much used in ecological studies. The mean annual temperature (MAT) is given in degrees Celsius times 10, the annual precipitation in mm, the temperature seasonality (Tseason) as 100 times the standard deviation of the monthly temperature, and the precipitation seasonality (Pseason) as the CV of monthly precipitation (given as a percent, i.e. times 100). Why is the temperature seasonality measured as a standard deviation and the precipitation as a CV?

The *effort* variable in the log number of hours of sampling, and the *method* variable indicates whether the collecting was done with hand nets, traps, or a combination of both.

Use a GLM to build a model explaining the distribution patterns of *Eulaema nigrita*. Interpret the results and produce nice tables and figures.

```
dat = read.csv("datasets/Eulaema.csv")
head(dat)
```

```
##   Eulaema_nigrita
```

```
## 1             492
## 2             372
## 3             679
## 4             600
## 5              28
## 6             535
##                                                                  SA  SU   method
## 1                                                  Morro_do_Itaoca 1-1 NetTraps
## 2                       Esta\xe7\xe3o_Ecol\xf3gica_de_Guaxindiba 2-1    Traps
## 3                       Esta\xe7\xe3o_Ecol\xf3gica_de_Guaxindiba 2-2    Traps
## 4                                                    Mata_do_Funil 2-3    Traps
## 5 \xc1rea_de_Prote\xe7\xe3o_Ambiental_da_Barra_do_Rio_Mamanguape 6-1      Net
## 6 \xc1rea_de_Prote\xe7\xe3o_Ambiental_da_Barra_do_Rio_Mamanguape 6-2      Net
##     effort altitude MAT  MAP Tseason Pseason    forest.    lu_het
## 1 4.189655        6 235 1073    2036      53 0.04416404 1.0531299
## 2 5.204007       17 230  987    1760      49 0.18217054 0.7571063
## 3 5.204007       17 230  987    1760      49 0.18217054 0.7571063
## 4 5.204007       30 231 1030    1820      51 0.01577287 0.8277580
## 5 4.969813        0 259 1693    1074      62 0.05000000 0.9672604
## 6 4.969813       43 255 1697    1061      63 0.05000000 0.8711333
```

```
m = glm(Eulaema_nigrita~effort+altitude+MAT, "poisson", data=dat)
```

Hurdle models

In biology we often have count data with many zeros. This is one of the causes of overdispersion as discussed above. Another way to deal with this is to split the analysis into two independent components, where the first models the zeros, and the second models the counts given that at least 1 entity was observed. This is called a hurdle model, where hurdle refers to the separation of zeros from non-zeros. We can fit a hurdle model in two parts and then combine the predictions.

Below we define two new response variables, where the first ($y1$) is a 0/1 variable, and the second ($y2$) has all the zeros set to `NA`.

```
y1 = ((y>1)*1)
m1 = glm(y1~x, family="binomial" (link="logit"))

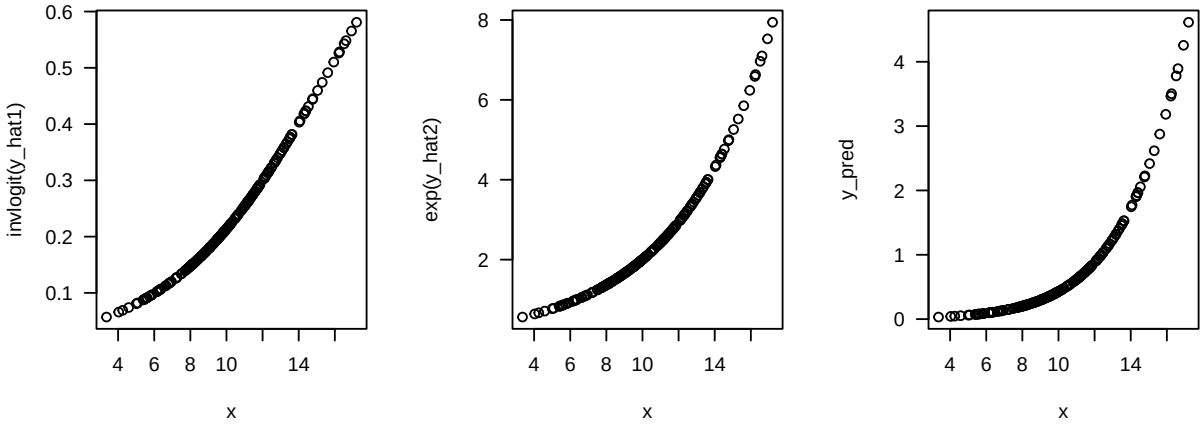
y2 = y
y2[which(y==0)] = NA

m2 = glm(y2~x, family="poisson", na=na.exclude)
```

```
coefs1 = summary(m1)$coef
coefs2 = summary(m2)$coef
y_hat1 = coefs1[1,1] + coefs1[2,1]*x
y_hat2 = coefs2[1,1] + coefs2[2,1]*x

y_pred = invlogit(y_hat1)*exp(y_hat2)

par(mfrow=c(1,3))
plot(x, invlogit(y_hat1), las=1)
plot(x, exp(y_hat2), las=1)
plot(x, y_pred, las=1)
```


invlogit(y_hat1)
0.6
0.5
0.4
0.3
0.2
0.1
4
6
8
10
14
x
exp(y_hat2)
8
6
4
2
4
6
8
10
14
x
y_pred
4
3
2
1
0
4
6
8
10
14
x